

Multiprocessing

July 24: ProcessPoolExecutor and MapReduce



Big idea

Use processes when pure Python CPU work needs multiple cores.



Student output

Build and benchmark a parallel preprocessing pipeline.

Dong Li
UC Merced

Today's Roadmap

From CPU bottlenecks to parallel feature extraction

01

Recap

threads vs
processes

02

Processes

memory, GIL,
lifecycle

03

Pools

ProcessPoolExecutor
and chunksize

04

Map- Reduce

chunk,
compute,
aggregate

05

Lab

parallel
preprocessing
benchmark

1. Recap and Motivation

Why Day 3 moves from threads to processes

Day 2 Recap: Threads Help Waiting

But CPU-heavy Python needs a different tool

T

Threads

Good for file/network waiting and producer–consumer pipelines.

P

Processes

Good for CPU-heavy Python work that can be split into independent tasks.

V

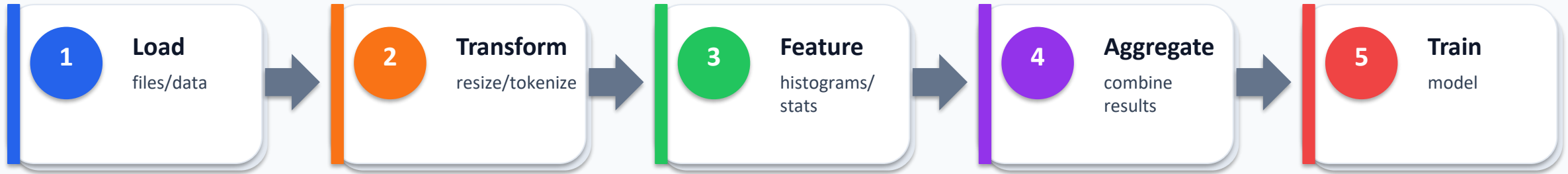
Vectorization

Often best for numeric arrays; use NumPy/PyTorch when possible.

Day 3 question: how do we split CPU work across multiple Python interpreters?

Motivation: CPU-Heavy Preprocessing

Feature extraction can dominate training time



serial preprocessing

CPU cores underused

process pool

Where Multiprocessing Appears in AI Workflows

Use it when Python-level preprocessing dominates

IMG

Image preprocessing

resize, filters, color stats,
histograms

TXT

Text processing

tokenization, n-grams,
vocabulary counts

CSV

Tabular ETL

chunk CSVs, feature engineering,
group stats

REC

Recommendation

similarity chunks, candidate
scoring

MET

Evaluation

metrics over large prediction
shards

HP

Hyperparameter trials

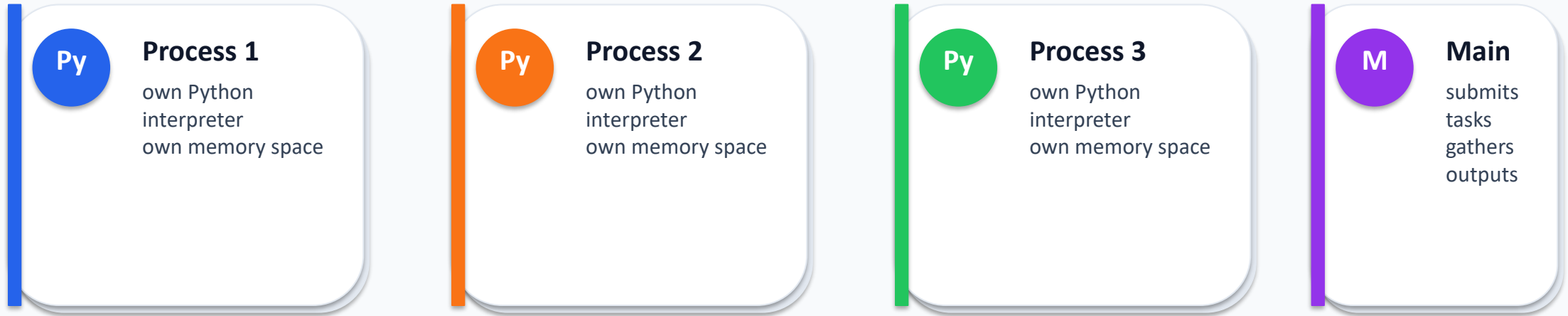
independent training jobs

2. Process Fundamentals

Separate interpreters, separate memory, true CPU parallelism

What Is a Process?

A separate Python interpreter with its own memory



Processes do not share normal Python objects by default — data must be sent or returned

Why Processes Help CPU-Bound Python

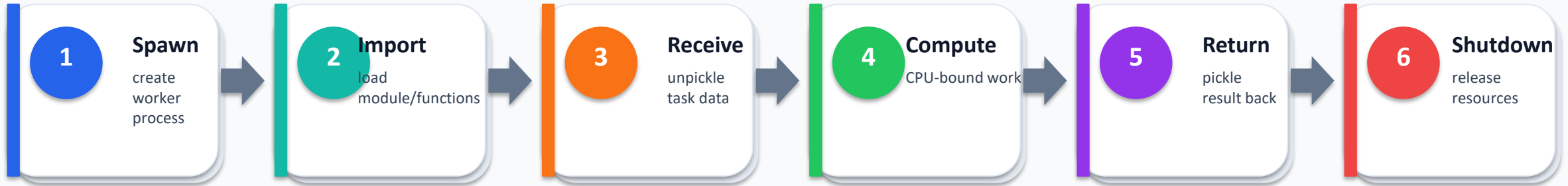
Each process has its own interpreter and GIL



Multiprocessing side-steps one shared GIL by using subprocesses instead of threads

Process Lifecycle

Spawn → initialize → compute → return → shutdown



Startup and serialization overhead are why tasks must be large enough to justify processes

Two Practical Rules

Most multiprocessing bugs come from these two issues

1

Rule 1: protect entry point

Put process-pool execution under:

```
if __name__ == "__main__":  
    main()
```

This avoids accidental recursive process spawning.

2

Rule 2: make workers picklable

Worker functions should be defined at the top level of the module. Avoid lambdas and nested functions for process tasks.

Two Practical Rules

Most multiprocessing bugs come from these two issues

1

Rule 1: protect entry point

Put process-pool execution under:

```
if __name__ == "__main__":  
    main()
```

This avoids accidental recursive process spawning.

2

Rule 2: make workers picklable

Worker functions should be defined at the top level of the module. Avoid lambdas and nested functions for process tasks.

```
a = 'GeeksforGeeks'  
upper = lambda x: x.upper()  
print(upper(a))
```

Example: Manual Process

Useful for understanding; process pools are usually cleaner

```
from multiprocessing import Process
import os

def worker(name):
    print(f"hello from {name}, pid={os.getpid()}")

if __name__ == "__main__":
    processes = []

    for i in range(4):
        p = Process(target=worker, args=(f"worker-{i}",))
        p.start()
        processes.append(p)

    for p in processes:
        p.join()

    print("all processes finished")
```

3. Process Pools and Performance

ProcessPoolExecutor, Pool.map, futures, and chunksize

Why Use a Process Pool?

Reuse worker processes and simplify scheduling

P

Manual processes

You create, start, join, and coordinate each process yourself.

POOL

Process pool

A fixed set of worker processes handles many tasks.

F

Futures/results

Main process submits work and collects outputs.

ProcessPoolExecutor is the high-level interface students should use first

Example: ProcessPoolExecutor.map

Good for one function applied to many independent inputs

```
from concurrent.futures import ProcessPoolExecutor
import os, time

def cpu_feature(x):
    total = 0
    for i in range(20_000):
        total += (x * i) % 97
    return total

def main():
    data = list(range(50_000))

    with ProcessPoolExecutor(max_workers=os.cpu_count()) as ex:
        features = list(ex.map(cpu_feature, data, chunksize=100))

    print(len(features), features[:5])

if __name__ == "__main__":
    main()
```


Example: submit + as_completed

Better for variable task sizes and robust error handling

```
from concurrent.futures import ProcessPoolExecutor, as_completed

def process_chunk(chunk):
    # CPU-heavy processing on one chunk
    return sum((x * x) % 97 for x in chunk)

def main():
    chunks = [list(range(i, i + 10_000)) for i in range(0, 100_000, 10_000)]

    with ProcessPoolExecutor(max_workers=4) as ex:
        futures = {ex.submit(process_chunk, c): idx for idx, c in enumerate(chunks)}

        partials = []
        for fut in as_completed(futures):
            idx = futures[fut]
            try:
                partials.append(fut.result())
            except Exception as e:
                print("chunk failed", idx, e)

    print("final result:", sum(partials))

if __name__ == "__main__":
    main()
```

Alternative: multiprocessing.Pool

Classic multiprocessing interface



```
from multiprocessing import Pool

def square(x):
    return x * x

def main():
    data = list(range(10))

    with Pool(processes=4) as pool:
        results = pool.map(square, data)

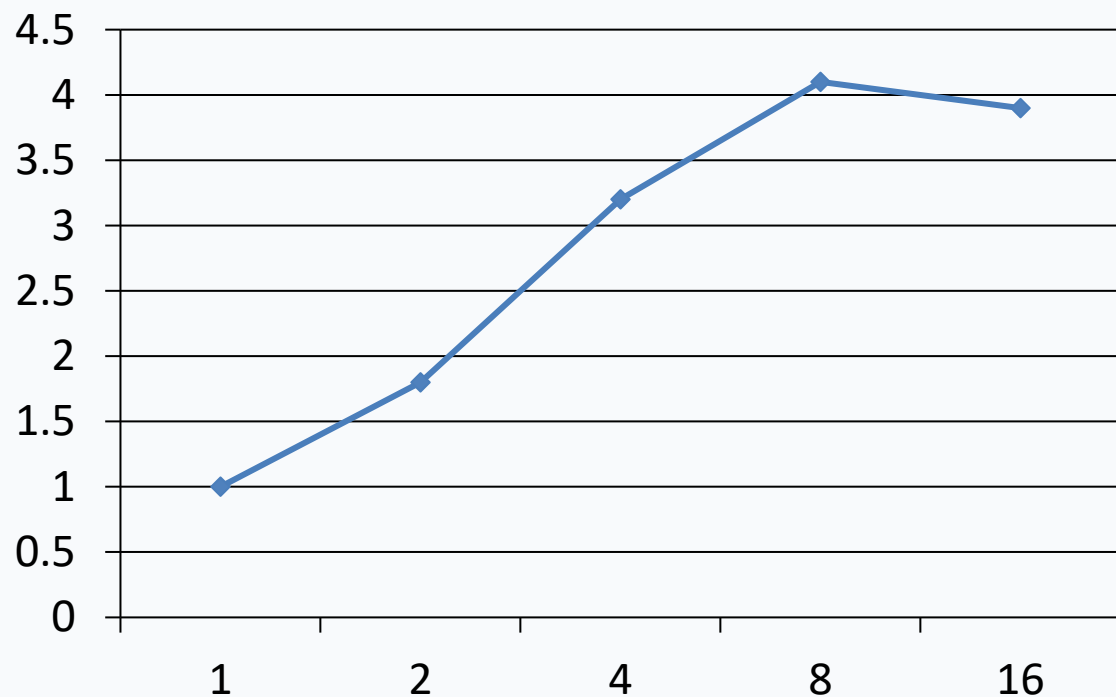
    print(results)

if __name__ == "__main__":
    main()
```

Tuning max_workers and chunksize

Performance is empirical: measure on your machine

Example CPU speedup vs workers



max_workers

Usually start near number of CPU cores.



chunksize

Bigger chunks reduce scheduling overhead.



too much

Too many workers can increase overhead and memory pressure.

Process Overhead Model

CPU parallelism has real costs

Total process-pool time $\approx$

pool startup cost
+ task scheduling cost
+ pickling input data
+ max(worker compute times)
+ pickling output data
+ final aggregation cost

Goal: make useful computation dominates overhead.

PKL

Pickling

data must be serialized between processes

RAM

Memory

each process may hold its own copy

Σ

Aggregation

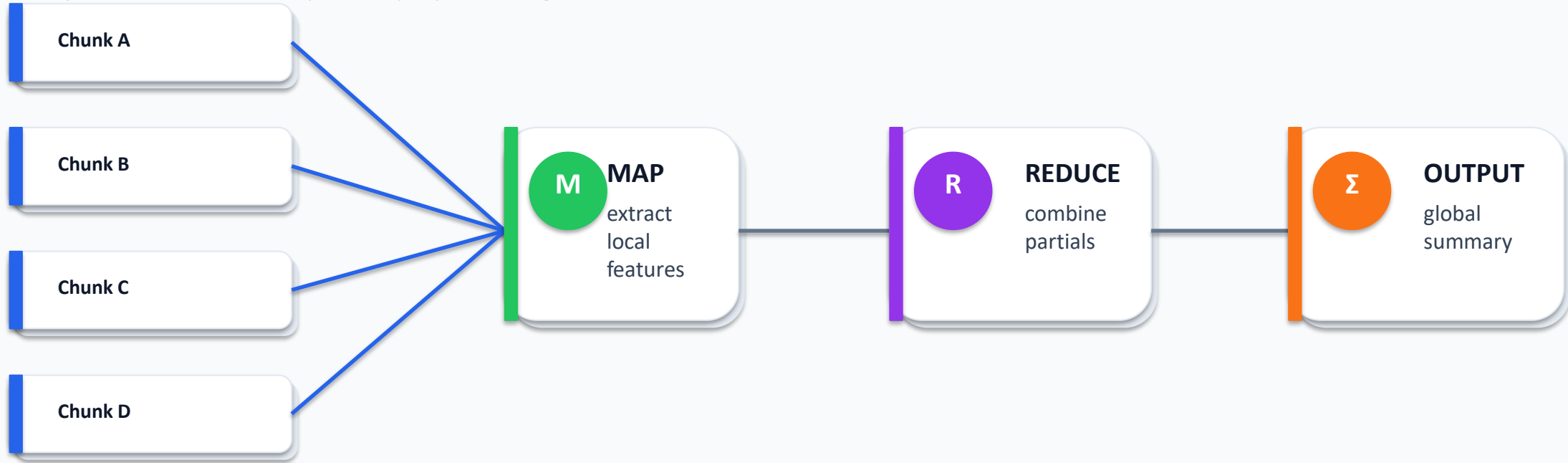
reduce partial outputs in main process

4. Map–Reduce Preprocessing

Split data, compute locally, aggregate partial results

Map-Reduce Pattern

A simple mental model for parallel preprocessing



Design workers to return small partial results; reduce in the main process

Three Lab Pathways

Students can choose the data type closest to their project

IMG

Image pathway

Map: compute per-image stats
Reduce: aggregate brightness, sizes, histograms
Extension: preprocessing for classifier

TXT

Text pathway

Map: tokenize chunk, count words
Reduce: merge Counter objects
Extension: vocabulary build

CSV

Tabular pathway

Map: compute chunk statistics
Reduce: combine sums/counts
Extension: large CSV ETL

Design Rule: Avoid Shared Mutable State

Make workers pure when possible



X

Risky style

Workers mutate shared counters, lists, or files. This requires locks and is easy to get wrong.



✓

Preferred style

Each worker receives input and returns a result. Main process aggregates outputs.

Pure functions are easier to test, parallelize, and reproduce

Common Multiprocessing Pitfalls

1

Nested functions

not picklable in many contexts

2

Huge inputs

serialization dominates runtime

3

Too-small tasks

scheduling cost dominates

4

No `__main__` guard

can recursively spawn processes

5

Too many workers

memory and context-switch overhead

6

Unclear baseline

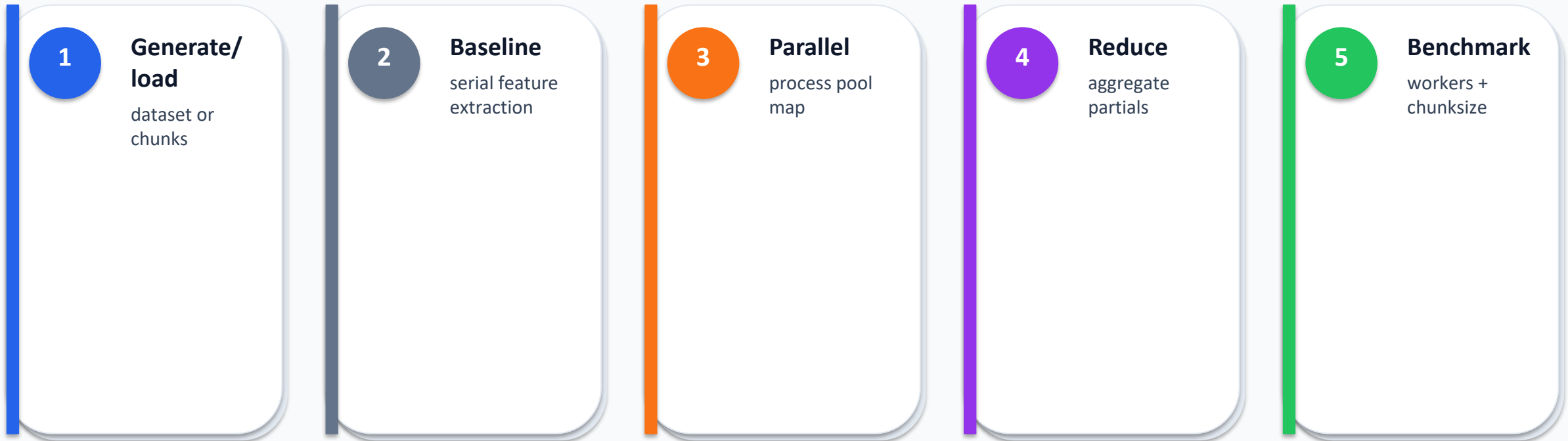
cannot prove speedup

5. Day 3 Lab

Build a parallel CPU preprocessing pipeline

Lab: Parallel Feature Extraction

Goal: speed up CPU-heavy preprocessing while preserving correctness



Deliverable: results table + speedup chart + explanation of bottlenecks

Lab Starter: CPU-Heavy Feature Function

A synthetic task that behaves like Python preprocessing



```
def cpu_feature(x):  
    Simulate CPU-heavy feature extraction.  
    total = 0  
    for i in range(30_000):  
        total += (x * i) % 97  
    return total  
  
def make_data(n=40_000):  
    return list(range(n))
```

Step 1: Serial Baseline

Measure simplest correct version first



```
import time

def run_serial(data):
    return [cpu_feature(x) for x in data]

def time_call(label, func):
    start = time.perf_counter()
    result = func()
    elapsed = time.perf_counter() - start
    print(f"{label}: {elapsed:.3f} seconds")
    return result, elapsed

data = make_data()
serial_result, serial_time = time_call("serial", lambda: run_serial(data))
```

Step 2: ProcessPool Parallel Version

Use chunksize to reduce scheduling overhead

```
from concurrent.futures import ProcessPoolExecutor
import os

def run_parallel(data, workers, chunksize=100):
    with ProcessPoolExecutor(max_workers=workers) as executor:
        return list(executor.map(cpu_feature, data, chunksize=chunksize))

def main():
    data = make_data()
    serial_result, serial_time = time_call("serial", lambda: run_serial(data))

    for workers in [1, 2, 4, os.cpu_count()]:
        parallel_result, elapsed = time_call(
            f"processes workers={workers}",
            lambda w=workers: run_parallel(data, workers=w, chunksize=100)
        )
        assert parallel_result == serial_result
        print("speedup:", serial_time / elapsed)

if __name__ == "__main__":
    main()
```

Step 3: Chunk-Level Map–Reduce

Return smaller partial results instead of huge output lists

```
from concurrent.futures import ProcessPoolExecutor

def process_chunk(chunk):
    partial_sum = 0
    partial_count = 0
    for x in chunk:
        partial_sum += cpu_feature(x)
        partial_count += 1
    return partial_sum, partial_count

def chunk_data(data, chunk_size):
    for i in range(0, len(data), chunk_size):
        yield data[i:i + chunk_size]

def run_map_reduce(data, workers=4, chunk_size=1000):
    chunks = list(chunk_data(data, chunk_size))
    with ProcessPoolExecutor(max_workers=workers) as ex:
        partials = list(ex.map(process_chunk, chunks))
    total_sum = sum(p[0] for p in partials)
    total_count = sum(p[1] for p in partials)
    return total_sum, total_count
```

Step 4: Benchmark Workers and Chunk Size

Create evidence for your design choice

```
def benchmark_grid(data):
    worker_counts = [1, 2, 4, 8]
    chunk_sizes = [100, 500, 1000, 5000]
    rows = []

    for workers in worker_counts:
        for chunk_size in chunk_sizes:
            start = time.perf_counter()
            total_sum, total_count = run_map_reduce(
                data, workers=workers, chunk_size=chunk_size
            )
            elapsed = time.perf_counter() - start
            rows.append({
                "workers": workers,
                "chunk_size": chunk_size,
                "time": elapsed,
                "items_per_sec": len(data) / elapsed,
                "total_sum": total_sum,
                "total_count": total_count,
            })
            print(rows[-1])
    return rows
```


Optional Pathway: Parallel Word Count

A classic map–reduce example for text data



```
from collections import Counter
from concurrent.futures import ProcessPoolExecutor

def count_words(lines):
    c = Counter()
    for line in lines:
        words = line.lower().split()
        c.update(words)
    return c

def merge_counters(counters):
    total = Counter()
    for c in counters:
        total.update(c)
    return total

def parallel_word_count(lines, workers=4, chunk_size=1000):
    chunks = [lines[i:i+chunk_size] for i in range(0, len(lines), chunk_size)]
    with ProcessPoolExecutor(max_workers=workers) as ex:
        partials = list(ex.map(count_words, chunks))
    return merge_counters(partials)
```

Mini Challenge

Optimize and explain your CPU pipeline

1

Run

Benchmark workers = 1, 2, 4, 8 and several chunk sizes.

2

Record

runtime, speedup, efficiency, items/sec, correctness.

3

Explain

Why does speedup flatten?
Where does overhead appear?

Stretch: compare item-level map vs chunk-level map–reduce

Exit Ticket

Short reflection + benchmark table

Q

Answer

- Why can processes speed up CPU-bound Python?
- What pickling rule did you follow?
- What worker count/chunk size worked best?
- Where did overhead appear?
- How could your project use map–reduce?

✓

Submit

- Code or notebook
- Runtime/speedup table
- Correctness check
- One chart
- 1-paragraph interpretation

References and Further Reading

Multiprocessing, process pools, and map–reduce thinking

1. Python multiprocessing documentation: process-based parallelism, Process, Pool, and side-stepping the GIL with subprocesses.

2. Python concurrent.futures documentation: Executor, Future, ProcessPoolExecutor, map, submit, and chunksize behavior.

3. Course connection: Day 4 parallel inference, Day 5 MPI-style scatter/gather/reduce, Day 6 data-parallel training.